

Tema 5 — Colecciones de objetos

Contenido Teórico

Objetivos

- **Identificar las colecciones principales de Python:** listas, tuplas, conjuntos y diccionarios.
- **Usar** índices, slicing, recorridos y operaciones habituales sobre colecciones.
- **Distinguir** mutabilidad, orden, duplicados y acceso por índice o por clave.
- **Aplicar colecciones a casos de entornos TI:** servicios, hosts, puertos, inventario, etiquetas y resultados.
- **Crear colecciones** por comprensión introduciendo el tema de programación funcional.
- **Elegir la estructura adecuada** según el problema y evitar errores frecuentes.

Mapa del Tema 5

- Partiremos de las secuencias ordenadas: listas y tuplas.
- Después estudiaremos conjuntos, útiles para valores únicos y operaciones de pertenencia.
- A continuación trabajaremos diccionarios, que asocian claves con valores y son esenciales para representar datos estructurados.
- Finalmente veremos colecciones anidadas, colecciones de objetos y comprensiones sencillas. El objetivo es saber organizar grupos de datos de forma clara antes de avanzar a programación funcional, errores, entrada-salida y módulos.

Qué es una colección

Una colección agrupa varios objetos bajo una misma variable. En Python una colección puede contener cadenas, números, booleanos, diccionarios, listas u objetos creados por nuestras clases.

- La elección de la colección afecta a cómo se accede a los datos: si se pueden modificar, si se permiten duplicados y si existe un orden estable.
- En scripts de entornos TI aparecen constantemente: listas de servicios, puertos permitidos, conjuntos de etiquetas, inventarios de servidores o respuestas estructuradas de APIs.

```
servicios = ["ssh", "nginx", "postgresql"]
puertos = [22, 80, 5432]

print(len(servicios))
print(servicios[0])
print(puertos[-1])
```

Cómo elegir una colección

- **La lista:** es la opción general cuando necesitamos una secuencia ordenada y modificable.
- **La tupla:** se usa cuando queremos una secuencia ordenada inmutable.
- **El conjunto:** sirve para garantizar valores únicos y comprobar pertenencia con rapidez.
- **El diccionario:** representa información mediante pares clave-valor. Es la estructura más natural para datos con nombre: host, IP, puerto, estado o usuario.

```
# Lista: ordenada y mutable
servicios = ["ssh", "nginx"]

# Tupla: ordenada e inmutable
puerto_https = ("https", 443)

# Set: únicos, sin índice
etiquetas = {"prod", "linux"}

# Dict: clave -> valor
servidor = {"host": "srv01", "ip": "10.0.0.20"}
```

Listas: secuencia ordenada y mutable

Una lista mantiene el orden de inserción, permite elementos repetidos y se puede modificar.

- Se crea con corchetes `[]` y sus elementos se separan con comas.
- Los índices empiezan en **0** y también pueden ser negativos para acceder desde el final.
- **Es la colección más flexible y probablemente la más usada al empezar:** permite almacenar resultados, tareas pendientes, rutas, servicios o líneas de un fichero.
- Aunque una lista puede ser heterogénea y combinar elementos de diferentes tipos, la convención es que todos los elementos deben ser del mismo tipo básico o compuesto (Strings, Enteros...)
- El equivalente en otros lenguajes es lo que se conoce como **arrays**.

```
servicios = ["ssh", "nginx", "postgresql"]

print(servicios)
print(type(servicios))
print(len(servicios))
print(servicios[0])
print(servicios[-1])
```

Listas: añadir elementos

Las listas son mutables: pueden crecer o cambiar durante la ejecución.

- Se puede crear una lista vacía declarándola con `[]` o invocando al constructor (`list()`)
- **`append()`** añade un elemento al final. **`insert()`** añade un elemento en una posición concreta.
- **`extend()`** añade varios elementos procedentes de otra colección.

Conviene elegir el método según la intención. Si solo queremos agregar un servicio al final, `append()` suele ser la opción más clara.

```
lista_vacia1 = []
lista_vacia2 = list()
print(lista_vacia1, lista_vacia2) # [] []

servicios = ["ssh", "nginx"]

servicios.append("postgresql")
servicios.insert(1, "firewalld")
servicios.extend(["chronyd", "rsyslog"])

print(servicios)
```

Listas: modificar y eliminar

- Un elemento puede modificarse asignando un nuevo valor en su índice.
- Para eliminar, podemos usar **`del`** si conocemos la posición o **`remove()`** si queremos borrar por valor.
- **`pop()`** extrae y devuelve un elemento, normalmente el último. Es útil cuando la lista funciona como una cola o pila sencilla.
- Si **`remove()`** no encuentra el valor, se produce **`ValueError`**, la gestión formal de errores llegará más adelante.

```
servicios = ["ssh", "nginx", "postgresql"]
print(servicios)

servicios[1] = "httpd"
print(servicios)

del servicios[0]
print(servicios)

servicios.remove("postgresql")
print(servicios)

servicios.append("rsyslog")
print(servicios)

ultimo = servicios.pop()
print(servicios)
print(ultimo)
```

Listas: slicing

El **slicing** obtiene fragmentos de una lista usando **[inicio:fin:paso]**.

- El índice inicial se incluye y el final no se incluye.
- **La sintaxis es la misma idea que vimos con cadenas**, pero aplicada a una lista de objetos.
- Es útil para tomar subconjuntos, omitir cabeceras, seleccionar los primeros resultados o recorrer una lista con saltos controlados.

```
hosts = ["srv01", "srv02", "srv03", "srv04", "srv05"]
print(hosts)

# Imprime desde el índice 0 hasta el índice 2
print(hosts[0:3])

# Imprime desde el inicio hasta el índice 1
print(hosts[:2])

# Imprime desde el índice 2 hasta el final
print(hosts[2:])

# Imprime los índices impares
print(hosts[::2])

# Imprime los índices en orden inverso
print(hosts[::-1])
```

Listas: recorrer elementos

Cuando solo necesitamos el valor, lo más sencillo es recorrer directamente la lista sin buscar el número de índice.

- Si necesitamos posición y valor, **enumerate()** evita construir índices manuales con **range(len())**.
- **NOTA:** técnicamente **enumerate()** es una clase que actúa como constructor. Sólo funciona dentro de una iteración y en cada paso te da el siguiente valor.
- El recorrido por índice sigue existiendo y es útil cuando necesitamos acceder a varias listas sincronizadas, pero debe usarse con criterio para no complicar el código.

En el ejemplo, si imprimes directamente **enumerate()** no funciona, una opción para verlo es convertirlo a lista (Lo veremos más adelante)

```
servicios = ["ssh", "nginx", "postgresql"]

for servicio in servicios:
    print(servicio)

for indice, servicio in enumerate(servicios, start=1):
    print(f"{indice}. {servicio}")

print(enumerate(servicios))
enumerate(servicios)
# Ambas muestran <enumerate object at 0x7facd71fccc0>
print(list(enumerate(servicios)))
# Muestra [(0, 'ssh'), (1, 'nginx'), (2, 'postgresql')]
```

Listas: buscar, contar y pertenencia

- El operador **in** comprueba si un elemento está en una lista. Es más legible que hacer una búsqueda manual.
- **index()** devuelve la posición de un valor, pero falla si no existe.
 - En código real conviene comprobar la pertenencia antes de usar **index()** cuando no estamos seguros de que el valor exista.
- **count()** cuenta el número de apariciones de un valor.

```
servicios = ["ssh", "nginx", "ssh", "postgresql"]

# Comprueba la existencia o no de un objeto
print("ssh" in servicios)
print("firewalld" not in servicios)

# Comprueba que existe un elemento antes de buscar index
if "nginx" in servicios:
    print(servicios.index("nginx"))

# Imprime el numero de ocurrencias.
print(servicios.count("ssh"))
```

Listas: ordenación y copia

- **sorted()** devuelve una nueva lista ordenada y deja la original intacta.
- **sort()** modifica la lista existente.
 - Para ordenar por un criterio concreto se puede usar **key** (Usa una función como **len** para el criterio de ordenación).
- **copy()** crea una **copia superficial** (Shallow Copy).
 - **Shallow Copy:** Python crea una nueva lista (un nuevo contenedor), pero no crea copias de los objetos que hay dentro. Simplemente copia las "direcciones" a esos objetos.
 - Si la lista contiene elementos inmutables (números, strings), si reasignas un número o una cadena en la copia, la lista original no se modifica.
 - **Cuidado:** Si la lista contiene elementos mutables (otras listas, diccionarios, objetos) Ambas listas apuntan al mismo objeto.

NOTA: En este tema basta con usar funciones simples como **len**, más adelante estudiaremos con más detalle el enfoque funcional.

```
servicios = ["nginx", "ssh", "postgresql", "api"]

# Muestra la lista ordenada (Orden alfabético)
print(sorted(servicios))
# La lista original permanece igual
print(servicios)

# Ordena la lista original por longitud de cadena
servicios.sort(key=len)
print(servicios)

# Hace una copia de la lista original
copia = servicios.copy()
copia.append("backup")
print(servicios)
print(copia)
```

Listas multidimensionales

Una lista puede contener otras listas: Esto permite representar tablas, matrices sencillas o grupos de datos relacionados.

- **Para acceder a un valor se encadenan índices:** primero la fila y después la columna.

En entornos TI puede usarse para datos tabulares pequeños, aunque para estructuras con nombres claros suelen ser más legibles los diccionarios.

```
estado_hosts = [  
    ["srv01", "OK", 22],  
    ["srv02", "FALLO", 80],  
    ["srv03", "OK", 443],  
]  
  
print(estado_hosts[0][0])  
print(estado_hosts[1][1])  
  
for fila in estado_hosts:  
    print(f"{fila[0]} -> {fila[1]}:{fila[2]}")
```

Listas de objetos

- Las listas pueden almacenar instancias creadas por nuestras clases. Esto conecta este tema con clases y objetos.
- Si varios objetos tienen los mismos métodos, podemos recorrerlos de forma uniforme y pedir a cada objeto su estado.
- **Este patrón aparecerá constantemente:** listas de servicios, tareas, usuarios, reglas o resultados de comprobación.

```
class Servicio:  
    def __init__(self, nombre, activo):  
        self.nombre = nombre  
        self.activo = activo  
  
    def comprobar(self):  
        if self.activo:  
            estado = "OK"  
        else:  
            estado = "DETENIDO"  
        return f"{self.nombre}: {estado}"  
  
servicios = [  
    Servicio("ssh", True),  
    Servicio("api", False),  
]  
  
for servicio in servicios:  
    print(servicio.comprobar())
```

Comprensión de listas

Una comprensión de listas (y otras colecciones) permite crear una nueva lista a partir de otra colección **usando una expresión compacta**.

- Por lo general combina en una expresión, un bucle **for** y, opcionalmente un condicional **if** y se define en una sola línea.
- **Es útil para transformaciones simples:** normalizar nombres, filtrar estados o extraer campos.
- **Debe usarse solo cuando mejora la lectura:** Si la lógica crece demasiado, un bucle for tradicional suele ser más claro para mantenimiento y soporte.

NOTA: El término correcto es **comprensión** (list comprehension) no confundir con compresión. En terminología matemática de definición de conjuntos existen dos formas posibles:

- **Definición por extensión:** Enumeras todos los elementos: {2,4,6,8}.
- **Definición por comprensión:** Das la regla que los define: {x es par y 0<x<10}.

```
servicios = [" SSH ", " NGINX ", " postgresql "]

normalizados = [s.strip().lower() for s in servicios]
print(normalizados)

puertos = [22, 80, 443, 8080, 3306]
puertos_web = [p for p in puertos if p in (80, 443, 8080)]
print(puertos_web)
```

Tuplas: secuencia ordenada e inmutable

Una tupla es una colección ordenada, con índice, pero inmutable: Una vez creada no se pueden modificar, añadir ni eliminar elementos internos.

- Se usa para datos que forman una unidad y no deberían cambiar: pares, coordenadas, configuraciones fijas o registros pequeños.
- La inmutabilidad reduce errores cuando queremos proteger la estructura de un dato durante la ejecución.

```
endpoint = ("srv-web-01", "10.0.0.20", 443)

print(type(endpoint))
print(endpoint[0])
print(endpoint[1])
print(endpoint[2])

# endpoint[2] = 8443 # Genera TypeError
```

Tuplas: creación y desempaqueado

Las tuplas se crean normalmente con **paréntesis ()** y valores separados por comas.

- Se puede crear una tupla vacía con **()** o usando el constructor **(tuple())**.
- Para una tupla de un solo elemento, la coma es obligatoria.
- El desempaqueado asigna cada posición a una variable. Es útil cuando una función devuelve varios valores o cuando un registro pequeño tiene posiciones conocidas.

- Debe usarse cuando el significado de cada posición sea claro.

```
tupla_vacia1 = ()
tupla_vacia2 = tuple()
print(tupla_vacia1, tupla_vacia2) # () ()
print(type(tupla_vacia1))

solo_uno = ("ssh",)
print(solo_uno)
print(type(solo_uno))

endpoint = ("srv-web-01", "10.0.0.20", 443)

# Desempaquetado
host, ip, puerto = endpoint
print(host)
print(ip)
print(puerto)
```

Tuplas: recorrido y pertenencia

Como las listas, las tuplas se pueden recorrer con **for** y admiten **len()**, **in**, **slicing**, **count()** e **index()**.

- La diferencia principal es que no tienen métodos para modificar su contenido.
- Son adecuadas cuando queremos una secuencia estable y ligera que represente valores relacionados.

```
puertos_estandar = (22, 80, 443, 5432)

print(len(puertos_estandar))
print(443 in puertos_estandar)
print(puertos_estandar[:2])

for puerto in puertos_estandar:
    print(f"Puerto permitido: {puerto}")
```

Tuplas generadas a partir de iterables

No existe una comprensión de tuplas con la misma sintaxis directa que las listas: Si escribimos una expresión entre paréntesis, Python crea una expresión generadora.

- Para obtener una tupla debemos pasar esa expresión al constructor **tuple()**.
- En este tema basta con reconocer la diferencia. Los generadores se relacionan con técnicas que se estudiarán más adelante.

```
puertos = [22, 80, 443, 8080]
print(type(puertos)) # list

# Esto no crea una tupla, crea un generador
gen = (p for p in puertos if p >= 100)
print(type(gen)) # generator

puertos_altos = tuple(p for p in puertos if p >= 100)
print(puertos_altos)
print(type(puertos_altos)) # tuple
```


Conjuntos: valores únicos sin índice

Un conjunto almacena valores únicos.

- **No permite duplicados y no se accede por índice.**
- Se crea con llaves {}, pero un conjunto vacío debe crearse con el constructor **set()**, porque de lo contrario, {} crearía un diccionario vacío.

Los conjuntos son útiles para eliminar duplicados, comparar grupos de elementos y comprobar pertenencia.

```
etiquetas = {"prod", "linux", "prod", "web"}

print(etiquetas)
print(type(etiquetas))
print("prod" in etiquetas)

set_vacio = set()
print(type(set_vacio))
```

Conjuntos: añadir y eliminar

- **add()** agrega un elemento al conjunto. Si ya existe, no se duplica.
- **discard()** elimina un elemento si existe y no falla si no está.
- **remove()** también elimina un elemento, pero genera **KeyError** si el elemento no existe.

En scripts de soporte suele ser más seguro usar **discard()** cuando no estamos seguros de la presencia del valor.

```
roles = {"web", "db", "nosql", "app"}

roles.add("cache")
roles.add("web")
print(roles)

roles.discard("db")
roles.remove("web")
# roles.remove("backup") # KeyError
print(roles)
```

Conjuntos: deduplicar y convertir

Una forma sencilla de eliminar duplicados de una lista es convertirla a **set**.

- Después se puede convertir de nuevo a lista si necesitamos seguir usando una secuencia.
- Hay que recordar que el conjunto no conserva un índice de acceso. Si el orden importa estrictamente, conviene tratar la solución con más cuidado.

```
hosts = ["srv01", "srv02", "srv01", "srv03"]

unicos = set(hosts)
# Elimina duplicados porque un set
# solo conserva valores únicos
print(unicos)
```

```
hosts_unicos = list(unicos)
# Al volver a lista no debemos
# asumir el orden original
print(hosts_unicos)

print("srv02" in unicos)
```

Operaciones de conjuntos

Los conjuntos permiten comparar grupos de datos de forma sencilla:

- **La intersección** obtiene elementos comunes.
- **La unión** combina sin duplicar.
- **La diferencia** obtiene elementos que están en un conjunto pero no en otro (El orden importa en este caso).

Estas operaciones encajan bien en entornos TI: puertos permitidos frente a detectados, usuarios esperados frente a activos o paquetes instalados frente a requeridos.

```
permitidos = {22, 80, 443}
detectados = {22, 8080, 443, 3306}

print(permitidos & detectados)    # comunes
print(permitidos | detectados)    # unión
print(detectados - permitidos)    # no permitidos
print(permitidos ^ detectados)    # diferencias no comunes
```

Comprensión de conjuntos

Una comprensión de conjuntos crea un **set** a partir de una expresión.

- Es útil cuando queremos transformar datos y quedarnos solo con valores únicos.
- Como en cualquier comprensión, si la lógica deja de ser clara, es preferible escribir un bucle tradicional para facilitar el mantenimiento.

```
logs = ["ERROR ssh", "OK nginx", "ERROR api", "ERROR ssh"]

servicios_error = {linea.split()[1] for linea in logs if linea.startswith("ERROR")}
# startswith() comprueba si la cadena empieza por "ERROR"
# split() convierte cada cadena en una lista. Ej. "ERROR ssh" -> ["ERROR", "ssh"]
print(servicios_error)

roles = ["WEB", "web", "Db", "db"]
roles_norm = {r.lower() for r in roles}
print(roles_norm)
```

Diccionarios: claves y valores

Un diccionario almacena pares clave-valor.

- No se accede por **índice**, sino por **clave**.
- **Las claves no pueden repetirse:** si se asigna una clave existente, se sobrescribe su valor.

Es una estructura esencial para representar datos con nombre: un servidor, una respuesta JSON, una configuración, un usuario o el resultado de una comprobación.

```
servidor = {  
    "hostname": "srv-web-01",  
    "ip": "10.0.0.20",  
    "activo": True,  
}  
  
print(servidor)  
print(servidor["hostname"])  
print(servidor["ip"])
```

Diccionarios: acceso Seguro

El acceso con corchetes falla con **KeyError** si la clave no existe.

get() permite obtener un valor sin provocar error y puede recibir un valor por defecto.

En scripts de soporte y automatización es frecuente usar **get()** cuando los datos pueden venir incompletos desde un fichero, API o inventario externo.

```
servidor = {"hostname": "srv01", "ip": "10.0.0.20"}  
  
print(servidor["hostname"]) # "srv01"  
print(servidor.get("ubicacion")) # None  
print(servidor.get("ubicacion", "sin asignar"))  
# Devuelve el valor por defecto "sin asignar"  
if servidor.get("ip"):  
    print(f"IP detectada: {servidor['ip']}")
```

Diccionarios: modificar datos

Los diccionarios son mutables. Podemos añadir una clave nueva, modificar una existente o eliminar una entrada.

- **update()** permite incorporar varios pares clave-valor.
- **del** elimina una clave concreta, pero falla si no existe.
- **pop()** elimina y devuelve el valor.

Estas operaciones permiten construir o actualizar datos estructurados durante la ejecución.

```
servicio = {"nombre": "nginx", "puerto": 80}  
print(servicio)  
  
servicio["activo"] = True  
servicio["puerto"] = 443  
print(servicio)  
  
servicio.update({"protocolo": "tcp", "seguro": True})  
print(servicio)  
  
puerto = servicio.pop("puerto")  
print(servicio)  
print(puerto)
```

Diccionarios: recorrer claves, valores e items

Al recorrer directamente un diccionario, Python devuelve sus claves.

- **keys()** devuelve las claves
- **values()** devuelve los valores
- **items()** devuelve pares clave-valor.
- **items()** suele ser la forma más clara cuando necesitamos imprimir o procesar tanto la clave como el valor.

```
servicio = {
    "nombre": "nginx",
    "puerto": 443,
    "activo": True,
}

# Recorrido directo: devuelve las claves
for clave in servicio:
    print(clave, servicio[clave])
print()

# keys(): devuelve las claves
for clave in servicio.keys():
    print("Clave:", clave)
print()

# values(): devuelve los valores
for valor in servicio.values():
    print("Valor:", valor)
print()

# items(): devuelve pares clave-valor
for clave, valor in servicio.items():
    print(f"{clave}: {valor}")
```

Diccionarios anidados

Un diccionario puede contener otros diccionarios o listas. Esta estructura permite representar inventarios o configuraciones más completas.

- El acceso se realiza encadenando claves.
- **Debe mantenerse legible:** si la estructura crece demasiado, conviene documentarla o convertir parte del diseño en clases, como se vio en el tema anterior.

```
inventario = {
    "srv-web-01": {"ip": "10.0.0.20", "servicios": ["ssh", "nginx"]},
    "srv-db-01": {"ip": "10.0.0.30", "servicios": ["ssh", "postgresql"]},
}

# Imprime un elemento
print(inventario["srv-web-01"]["ip"])

# Recorre la estructura
for host, datos in inventario.items():
    print(host, datos["servicios"])
```

Diccionarios de objetos

También podemos almacenar objetos dentro de un diccionario: La clave identifica el objeto y el valor contiene la instancia.

- Esto resulta útil cuando queremos recuperar rápidamente un elemento por nombre, identificador, hostname o código.

La estructura del ejemplo combina lo aprendido antes con acceso por clave propio de los diccionarios.

```
class Servicio:
    def __init__(self, nombre, activo):
        self.nombre = nombre
        self.activo = activo

    def comprobar(self):
        if self.activo:
            return "OK"
        return "DETENIDO"

servicios = {
    "ssh": Servicio("ssh", True),
    "api": Servicio("api", False),
}

print(servicios["ssh"].comprobar())
print(servicios["api"].comprobar())
```

Comprensión de diccionarios

Una comprensión de diccionarios crea pares clave-valor a partir de una colección.

- Es útil para crear diccionarios de forma rápida a partir de listas o tuplas, evitando escribir bucles más largos.
- Como en listas y conjuntos, debe usarse cuando la expresión sea sencilla y se entienda de un vistazo.

```
servicios = ["ssh", "nginx", "postgresql"]
puertos = [22, 80, 5432]

mapa_puertos = {servicios[i]: puertos[i] for i in range(len(servicios))}
print(mapa_puertos)

longitudes = {servicio: len(servicio) for servicio in servicios}
print(longitudes)
```

Revisión: Mutabilidad y referencias

Cuando una colección contiene objetos mutables, la copia superficial (Shallow Copy) copia la estructura externa, pero no duplica automáticamente los objetos internos.

- Esto puede sorprender si modificamos un elemento interno desde una copia.

No hace falta profundizar todavía en copias avanzadas, pero sí conviene entender que las colecciones almacenan referencias a objetos.

```
original = [{"srv01", "OK"}, {"srv02", "FALLO"}]
copia = original.copy()

copia[0][1] = "REVISAR"

print(original)
print(copia)

# La lista interna es el mismo objeto referenciado.
```

Buenas Prácticas y Errores frecuentes con colecciones

Buenas Prácticas:

- Elegir la colección según el tipo de acceso: índice, clave, pertenencia o unicidad.
- Usar **get()** cuando una clave pueda no existir.
- No asumir orden en conjuntos.
- No modificar una colección mientras se recorre sin una estrategia clara.

Errores Frecuentes:

- Acceder a un índice inexistente
- Usar una clave que no existe
- Esperar orden en un conjunto
- Confundir **append()** con **extend()**
- Olvidar que las tuplas son inmutables
- Usar una lista multidimensional cuando un diccionario sería más claro.

```
servicios = ["ssh", "nginx"]
# print(servicios[5])          # IndexError

puertos = {"ssh": 22}
# print(puertos["web"])       # KeyError
print(puertos.get("web", "sin puerto"))

etiquetas = {"prod", "linux"}
# print(etiquetas[0])         # TypeError
```

Resumen del Tema 5

- Las listas son secuencias ordenadas y mutables.
- Las tuplas son secuencias ordenadas e inmutables.
- Los conjuntos almacenan valores únicos y permiten operaciones de comparación entre grupos.
- Los diccionarios organizan información mediante claves y valores, y son esenciales para datos estructurados.
- Las colecciones pueden contener objetos, anidarse y crearse mediante comprensiones sencillas.

En el siguiente tema aprovecharemos estas estructuras para introducir elementos de programación funcional.

```
def resumen_coleccion(tipo):  
    tipos = {  
        "list": "mutable",  
        "tuple": "inmutable",  
        "set": "valores únicos",  
        "dict": "clave-valor",  
    }  
    return tipos.get(tipo, "no revisado")  
  
print(resumen_coleccion("dict"))
```

Flujo de trabajo en el laboratorio

- Desde el repositorio Git podrás cargar el directorio del Tema05.
- El laboratorio reforzará el conocimiento sobre las diferentes colecciones de objetos, incluyendo creación, acceso, modificación y recorrido. Después se combinarán colecciones con clases, objetos y datos estructurados.
- La versión de Python usada en el venv del curso será Python 3.13.

```
# Cargar el entorno  
cd ~/Curso_Python  
source .venv/bin/activate  
python --version  
# Actualizar del repositorio Git  
git pull origin main  
# Para ejecutar VSCode  
code . # usar carpeta Tema05  
# Para ejecutar en Terminal  
python
```